

Stream Decoding — An Engineering Walkthrough

The same material as the academic monograph `DecoderGuide.Academic.en.md`, minus formalism for its own sake: shorter notation, more "why it is built this way" and "how it behaves on a real stream". Section numbering matches in both versions, so the cross-references are interchangeable. The from-scratch tutorial is `DecoderGuide.Tutorial.en.md`; context is in `AlgorithmGuide.en.md` (sections 5–9); the forward half of the codec is `EncoderGuide.Engineer.en.md`.

1. What we are taking apart

`FileDecoder` takes a stream (Base64 text, raw bytes, a `Stream`, a file) and turns it into reception slots — one per recognized file. Assembling from a slot is a separate shop (`AssemblyGuide.*`); here it is only the delivery pipeline: source → Base64 filter (txt) → sliding-window scanner → accumulator. Two scanners (txt/bin) live simultaneously and retain the data they have passed between `Scan` calls.

2. Pipeline architecture

Everything is built on the `IDataSource` / `IDataProcessor` interface pair (the `DataShield.Interface` assembly): a source reads blocks and announces `DataReady` with a take delegate; a processor `Attaches` to the source, does its work, and is itself a source for the next stage. `DataProcessorBase` provides: an output buffer with a delivery threshold, two output modes (byte `Emit` and packet `EmitPacket` — packets are never cut by delivery boundaries), cascading `Start/Stop/Complete`, all under `SyncRoot`. The `Error` channel: a source's exception bubbles up by delegation and `Completion` faults with that same error — I/O failures are not swallowed.

3. Input formats

`Scan(IEnumerable<string>)` — the lines are concatenated, UTF8, then the txt pipeline; shorter than 100 characters — a silent exit. `Scan(byte[])` — binary input; shorter than 75 — silent. `Scan(Stream, OutputFormat)` — explicit format; the progress length is taken from seekable streams. `PacketIO.ScanFile` looks at the extension: `.txt` → Base64, `.bin` → Binary, anything unrecognized → Base64.

4. The Base64 filter

`ByteRangeFilter.CreateBase64()` — a `bool[256]` table; passes `A-Z`, `a-z`, `0-9`, `+`, `/`, `=` is not needed: a packet line is exactly 100 characters with no padding. Line breaks, decorative framing, and garbage outside the alphabet are discarded before the scanner. In binary mode there is no filter in the chain.

5. The sliding window: forward pass

`SlidingWindowScanner(windowSize, handler)`: a window of 100 (txt) or 75 (bin); the `handler` returns an advance and optionally a packet. Failure → a 1-byte shift (a packet start cannot slip through); success → the packet is emitted and the window jumps by its size (the fast lane). The scanner **retains the entire stream** (`_retained`) — the price of rebinding (section 10). Progress goes out via the `ConsumedAdvanced` event at the forward-pass position.

6. Packet recognition

`FileDecoder.TxtWindow`: 100 characters → `Convert.TryFromBase64Chars` → 75 bytes; a decode failure or non-recognition → shift 1. `BinWindow`: raw 75 bytes straight to the check. `StreamProcessor.Recognizes` decides: a header — the autonomous `VerifyHeaderPacket` check (H5 over the first 51 bytes); a sector — iterating the slots: the number within `0..T-1` and `VerifySectorPacket(packet, slot.HeaderHash)`. False positives: 2^{-192} per header, 2^{-72} per sector.

7. The accumulator: chunks → packets → slots

`StreamProcessor.ProcessChunk`: a 75-byte `_pending` buffer assembles packets out of arbitrarily bounded chunks. A complete packet: header → `AcceptHeader` (byte-wise comparison against the slots: a match → `IncrementHeaderCount`; a newcomer → `HeaderContent.ReadFrom` + H5 + a new `ReceptionSlot`, with the `HeaderAccepted` event raised after the lock is released); a sector → into every fitting slot (`SectorMatches` → `AddSector(num, payload copy)`). Metrics out: `FileCount`, `TotalReceivedSectorCount`, `TotalReceivedSectorCopyCount`, `TotalCollisionSectorCount`.

8. Slots and versions

`ReceptionSlot` is a dictionary "number → version list by descending counter". All the version, collision, and assembly logic is `AssemblyGuide.Engineer.en.md`. One thing matters for the pipeline: a sector fitting several slots lands in each (files have different H5s; a coincidence is practically impossible).

9. Multi-file support

There are as many slots as there are distinct headers in the stream. Every sector is checked against all slots; confirmations are separate. `Slots` is a snapshot under a lock and can be read concurrently with reception.

10. Targeted rebinding

The problem: a sector before the first header is unrecognizable (no seed). The solution: on the `HeaderAccepted` event the decoder asks both scanners to rebind the retained data — a re-pass with `RebindWindow`: decode (txt), the number within the new `T`, `VerifySectorPacket` with the new `H5`. The active scanner executes the request deferred (the bound is `_flushedPos`); an idle one — immediately over the whole traversed extent, with the accumulator temporarily attached to it. The rebinding is targeted: other slots are untouched.

11. The rescan

Two hard rules of the re-pass. First — exhaustiveness: the window is checked at **every** position of the region, with no jumps; the forward pass jumps by the window after a success and, on damage, can leap over an overlapping packet — the re-pass closes those holes. Second — the boundary: rebinding goes only up to the forward pass's delivery position (`_flushedPos`, fixed in `OnDelivering` **before** the handlers run), otherwise confirmations would be doubled. Requests arriving during a pass are queued and executed after it.

12. Pipeline finalization

`RunPipeline` closes the chain in order: `filter?.Complete()` → `scanner.Complete()` → `processor.Complete()`. The scanner's final delivery raises the boundary and launches the deferred rebinds — covering the forward pass's omissions. Skipping `Complete` = lost tails and missing rebinds; reception completeness is guaranteed only after it.

13. Progress and cancellation

The phase: `HeaderSearch` while `FileCount == 0`, then `SectorSearch`; the percent is consumed/total via `ConsumedAdvanced`; the terminal is `Done` (100). Cancellation: checked in the progress callback and while awaiting `Completion`; the exception goes outward, the accumulated state survives.

14. Assembly

`TryAssemble(HeaderContent)`: serialize the header → a byte-wise slot search → `slot.TryAssemble(_rs, progress, ct)`. The RS adapter is created once for the decoder's lifetime. Beyond that — `AssemblyGuide.Engineer.en.md`.

15. The I/O module and errors

Sources: `ByteArraySource`, `StreamSource`, `FileSource`. Result sinks: `StreamDataWriter`, `FileDataWriter`, `ByteListWriter`, `PreallocatedBufferWriter`. The error channel: `BufferedSourceBase` catches a read exception, records it in `Error`, stops, and `Completion` faults; processors delegate `Error` upward. The result: a disk failure reaches the caller as an honest exception out of `Completion.Wait`, not as an empty result.

16. Guarantees and limits

Guaranteed:

1. Recognition only by an exact hash: garbage/forgeries do not enter the slots.
2. The step-1 rule on failure + the exhaustive re-pass: correct packets are not lost to garbage and desynchronization.
3. A sector-before-header is picked up by the rebinding; confirmations are not doubled (the delivery boundary).
4. I/O errors are not lost (`Error` / `Completion`).
5. Slot state is read thread-safely concurrently with reception.

Limits: scanner memory grows linearly with the input (the whole stream is retained — the price of rebinding); the cost of one spoiled Base64 character is up to ~99 window shifts; rebinding covers only the data retained during the decoder's current lifetime.